

Business Process Technologies and Management

5. Logging and Monitoring

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de



Differences



Logging:

- Available ex-post
- Data stored in files or databases
- Integrated into individual pieces of your architecture - i.e., each web-service or each software component logs individually.

Monitoring:

- For observing run-time.
- Data-streams; Potentially aggregated from individual pieces of your architecture.
- Structured as events:
 - Basics: timestamp, event-type, event name, life-cycle information, event content
 - Each component might deliver different events: in a different granularity, a different velocity, and with a different event content format.

Commonalities - Observability

Health checks - is the service working well?

- ping, heartbeat, resource utilization

Trace exchanged messages

- each message exchange has timestamp
- each message exchange has a unique id

Metrics - convey performance info, or inner state

- timing, memory consumption, storage consumption

Exception tracking

- errors in code

Auditing - strictly about business logic

- KPIs
- Exception tracking - exceptions in business logic

Logs 1



- Logs have a specific format:
 - NCSA Common Log Format https://en.wikipedia.org/wiki/Common_Log_Format
 - IEEE 1849-2016 XES Standard <https://xes-standard.org/>
- Logs have a file type:
 - XML (extendible markup language), JSON (javascript object notation):
 - not very well suited
 - each event has to be inserted at a certain point of the file (in contrast to beginning/end)
 - </end> and } prevent append to file
 - Bad IO performance (load, seek, insert, save), or memory usage (load once, insert, save, insert, save, ...)
 - CSV (character separated vector), YAML (yet another markup language), plain text
 - well suited
 - each event can be inserted at the end of a file
 - direct support by the OS/file system; highest possible performance (faster than database)

Logs 2



Logs have to be collected, unified, connected, and enriched

- collection: from different places/sources, e.g. databases, files
- unification: transformation into a common format
- connection: through timestamp, or better through semantic knowledge of data artefacts; i.e. an order id, ...
- enrichment: with order data, customer data, ...

Logs have to be securely stored/archived - Security

- economic espionage
- data leaks
- falsification; i.e. falsified production logs may lead to voided warranty and financial penalties

Logs contain sensitive data - Privacy

Unified, connected and enriched logs are the basis for Business Intelligence.

Logs 3 - Collect, Unify, Connect



ELK stack:

- Elasticsearch as a database for storing the logs.
- Logstash to read different file formats and deliver them to Elasticsearch.
- Kibana to do simple visualization and exploration of data.

Celonis: same idea, different visualization capabilities, more integrated.

Apache Flume: same idea, less polished data analysis and visualization capabilities.

Siemens Mindsphere: same idea, more adapters for industrial hardware and formats.

GE Predix: same idea, used in the US.

Microsoft PowerBI: same idea, works best when you have a certain infrastructure.

Logs 4



Security:

- Depends on software stack
- Fingerprinting individual messages
- Store in blockchain

Privacy:

- Depending on domain; i.e. medical domain (patients) may have higher requirements than manufacturing (workers).
- Data transformation: anonymisation, pseudonymisation (pseudonym translation table available)
- Specialized storage:
 - k-anonymity removes identifiers and pre-groups data
 - differential privacy adds additional artificial data, so that the data analysis is not skewed, but it becomes hard to identify individual subjects

Logs 5 - Analysis & Enrichment



All software stacks allow for standardized analysis:

- Timing analysis is simple
- Analysing semantic coherences always requires custom:
 - extraction of semantic information, e.g. data about quality of a product
 - configuration/code that interpretes/visualizes the information

Enrichment - the more information, the better the analysis:

- No free lunch - specialized code
- Examples:
 - independent sensor streams like temperature, location updates, ...
 - information about orders which have to be extracted from CRM (customer relationship management), ERP (enterprise resource planning), or MES (manufacturing execution system). All this information is probably highly domain specific, stems from different sources (SAP, ...) and has custom formats and information granularity

Logs 6 - Business Logic



How do the stacks handle the underlying business logic/process?

- The process is something that is independent from the log - many components or hardware contribute to the log.
- Mindsphere, PowerBI and Predix force you to recreate the process, i.e. configure how everything is connected together.
- Celonis gives you tools to simplify this recreation, and detect changes.
- All the others are more data-centric (process agnostic); manual work / custom code is necessary.

MAINTENANCE NIGHTMARE: changes to the underlying business logic/process require changes in logging/analysis stack.

Monitoring 1



Works on all information that is contained in a log, **BUT at runtime.**

Relies on data-streams - nothing is ever stored, it is sent through a network connection.

Data streams:

- PubSub
- Push/Pull
- No common / standardized format for the delivered data
- Common protocols: HTTP, MQTT (popular in IoT space).
- Common message types for individual events: Bson (Binary Json) & MsgPack (high performance), XML, YAML, ...
- Connected to enterprise service busses or message routing frameworks (MRF) like Apache Camel, Apache Kafka

Monitoring 2



Data streams ctd:

- Collection, Unification, Connection and Enrichment of individual data streams into one data stream has the same challenges as for logs: custom configuration and code.
- Security: secure protocols.
- Privacy: same challenges as for logs.
- Analysis:
 - Pipeline: e.g. Apache Kafka
 - Visualization - graphs that change at runtime to reflect:
 - the current state of a variable (e.g. stock of raw material)
 - a sliding window (e.g. quality of the last 100 produced products)
 - data drifts (e.g. difference in quality inside a batch of produced products)

Monitoring 3 - Software Stacks



Many of the before mentioned products are bad at monitoring, because they introduced delays and are not particularly well suited for high velocity (UI problem). Celonis, Mindsphere try to tackle the problem (please revisit the SOTA from time to time :-)

- Option 1: ELK - best of the rest for now. Kibana can not be tailored very well, so only for simple things. You probably have to write your own UI.
- Option 2: NASA OpenMCT - you have to bring your own stream analysis code/pipeline, but it helps you with UI.

Use your own message framework, such as ZeroMQ (simple and fast) or Apache Kafka (full featured and complicated) for best results.

Logging vs. Monitoring - Best Practice



Use Both concurrently!

Logging:

- Log only internal information, that helps you fix/debug software related problems in individual modules/services, i.e., exceptions and errors in your code.
- Don't log any business logic / process related information.
- Use highly localized logging so you don't have to enrich or do complex analysis.
- Any stack can be used, even simple and free ones such as ELK.

Monitoring:

- Monitor everything that has to do with your business logic and your processes.
- Will require lots of custom transformation, and connection anyway (e.g. custom machine learning cases).
- Think about feeding the results back into your system (i.e., self-controlling systems) for continuous improvements.

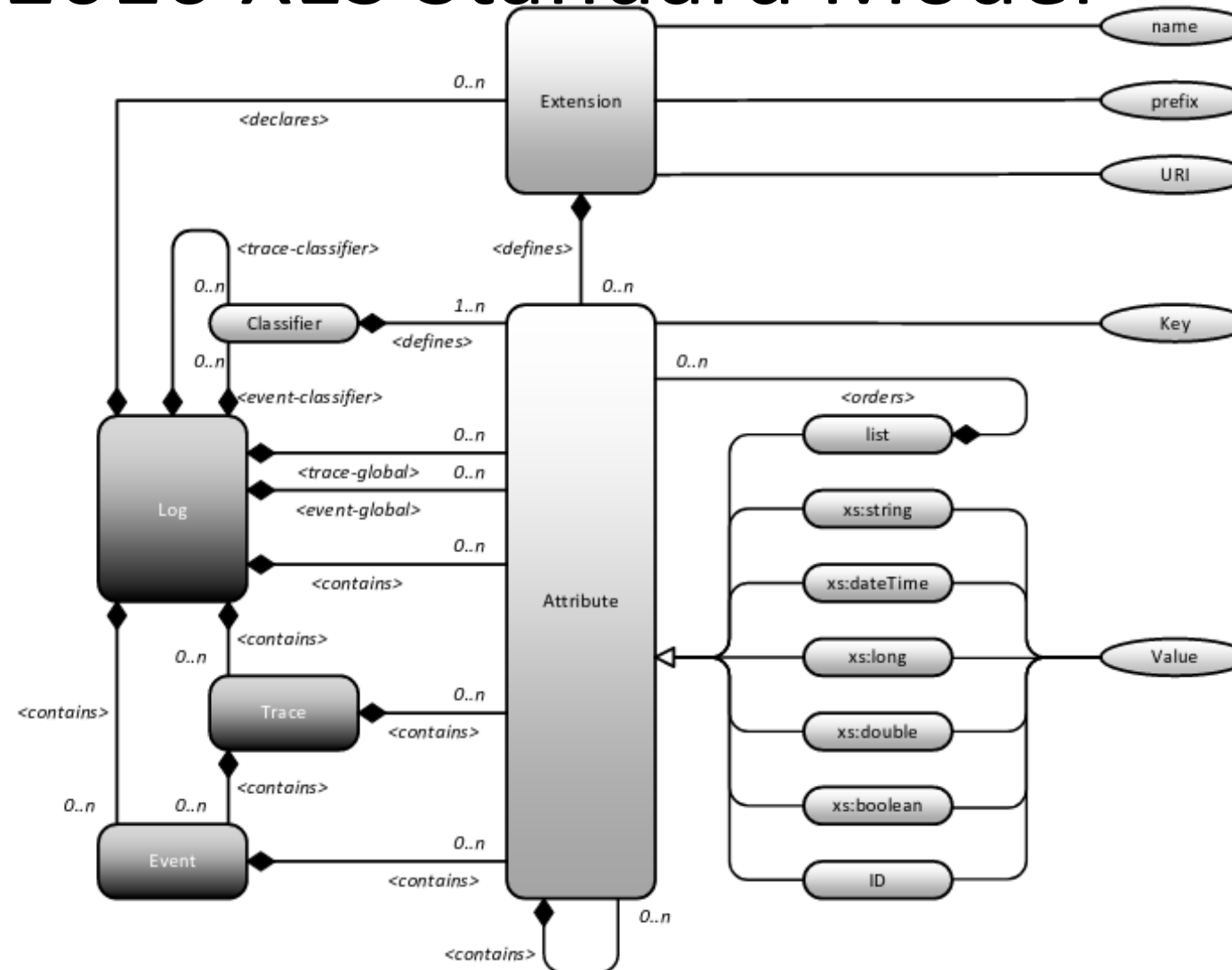
Logging vs. Monitoring - BPM



Use Process Management Engines!

- They separate business logic / processes from functionality / service implementations.
- Functionality / service implementations may log locally.
- The process engine will automatically create a unified, connected, and enriched data stream.
- Effort for custom analysis is the same, but maintainability is much better:
 - No need for custom duplicated code / configuration to collect, unify, and connect data.
 - Enrichment is part of the process model.
 - Analysis is much simpler and straight forward as the analysis pipeline is much shorter.

IEEE 1849-2016 XES Standard Model



IEEE 1849-2016 XES Standard Header



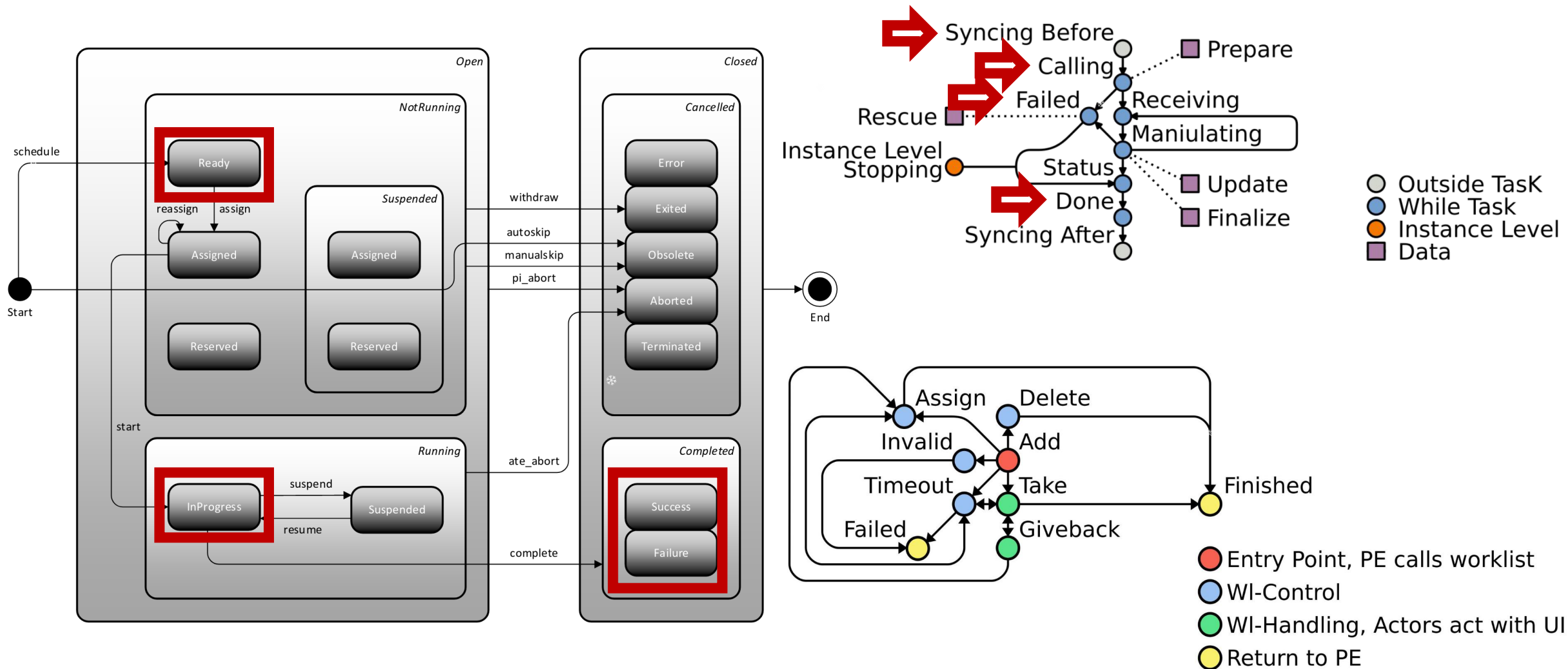
```
<log xes.version="1.0" xmlns="http://code.deckfour.org/xes" xes.creator="Fluxicon Nitro">
  <!-- Extensions describe possible attributes and their data types -->
  <extension name="Concept" prefix="concept" uri="http://code.deckfour.org/xes/concept.xesext"/>
  <extension name="Time" prefix="time" uri="http://code.deckfour.org/xes/time.xesext"/>
  <extension name="Organizational" prefix="org" uri="http://code.deckfour.org/xes/org.xesext"/>
  <global scope="trace"> <!-- Default values for attributes -->
    <string key="concept:name" value=""/>
  </global>
  <global scope="event"> <!-- Structure of events , a set of attributes -->
    <string key="concept:name" value="string"/>
    <string key="identifier:id" value="a0"/>
    <string key="lifecycle:transition" value="unknown"/>
    <date key="time:timestamp" value="1970-01-01T00:00:00.000+01:00"/>
    <list key="parameters">
      <string key="org:resource" value="string"/>
      <string key="Activity" value="string"/>
      <string key="Resource" value="string"/>
      <string key="Costs" value="string"/>
    </list>
  </global>
  <!-- One trace per instance, see lifecycle of instances in Chapter 3 of this lecture -->
  <trace></trace>
  <trace></trace>
</log>
```

IEEE 1849-2016 XES Standard Trace



```
<trace>
  <string key="concept:name" value="Instance 3"/>
  <string key="creator" value="Fluxicon Nitro"/>
  <event>
    <string key="concept:name" value="register request"/>
    <string key="identifier:id" value="a1"/>
    <string key="lifecycle:transition" value="start"/>
    <date key="time:timestamp" value="2010-12-30T14:32:00.000+01:00"/>
    <list key="parameters">
      <string key="org:resource" value="Pete"/>
      <string key="Activity" value="register request"/>
      <string key="Resource" value="Pete"/>
      <string key="Costs" value="50"/>
    </list>
  </event>
  ...
</trace>
```

XES Lifecycle vs. CPEE Lifecycle



XES Lifecycle vs. CPEE Lifecycle



In XES not much is about tasks, major focus on resources (i.e. worklists). Different POVs for worklists:

- XES: Worklist is a part of the process engine and its lifecycle.

VS.

- CPEE: Worklist is just another task, each task can have its own internal lifecycle.

CPEE Data Streams



PubSub, pushes data to HTTP endpoints.

Topics to subscribe to (<https://cpee.org/flow/engine/1/notifications/topics/>):

- Activity: see lifecycle in Chapter 3 of this lecture. Events are sent whenever an activity is called.
- Condition: Events are sent whenever a condition is evaluated.
- Position: Events are sent whenever a task is finished and the thread of control moves on to another task.
- Description: Events are sent whenever the process model is changed.
- State: see lifecycle in Chapter 3 of this lecture. Events are sent whenever the process engine starts/stops/finishes.
- Data elements: Events are sent whenever the process context changes.

CPEE Data Streams



- Endpoints: Events are sent whenever the list of functionalities invoked by a process model/instance changes.
- Attributes: Events are sent whenever attributes of the process model/instance change, e.g. a new author.
- Task: Events are sent whenever a functionality / service implementation wants to communicate internal lifecycle transitions through the process engine (i.e. lifecycle transition forwarding through the process engine).

CPEE Data Stream Subscriptions 1



Each process instance can have many subscriptions. The subscription format is as follows:

```
<subscriptions xmlns="http://riddl.org/ns/common-patterns/notifications-producer/2.0">
  <subscription id="_student" url="https://lehre.bpm.in.tum.de/~tumuser/log.php">
    <topic id="state">
      <event>change</event>
    </topic>
    <topic id="activity">
      <event>calling</event>
      <event>failed</event>
      <event>done</event>
    </topic>
  </subscription>
</subscriptions>
```

CPEE Data Stream Subscriptions 2



Each subscription has:

- an id: e.g. _student
- an endpoint where to push the messages
- a set of topics and events to be pushed

Subscription endpoints might be:

- Apache Kafka or other analysis frameworks
- Custom code for logging or visualization

Summary



Logging should deal with localized errors, **monitoring** should be about making the business logic / process observable.

Logs are stored in standardized log formats such as IEEE 1849-2016 XES.

- Logs have to be collected, unified, connected and enriched for ex-post for analysis.

Monitoring relies on data streams either from:

- Individual software components - these data streams have to be unified, connected and enriched before analysis.
- Process Engines - these data streams can be directly analysed.

Monitoring shows what is going on while it is going on.